

Lecture 10: Crash Recovery

CS211 - Operating systems

Spring 2026



Transaction

A transaction is an atomic sequence of reads and writes that takes the system from consistent state to another

"Classic" example

```
BEGIN;      --BEGIN TRANSACTION
UPDATE accounts SET balance = balance - 100.00 WHERE
    name = 'Alice';

UPDATE branches SET balance = balance - 100.00 WHERE
    name = (SELECT branch_name FROM accounts WHERE name
        = 'Alice');

UPDATE accounts SET balance = balance + 100.00 WHERE
    name = 'Bob';

UPDATE branches SET balance = balance + 100.00 WHERE
    name = (SELECT branch_name FROM accounts WHERE name
        = 'Bob');

COMMIT;      --COMMIT WORK
```

Transfer \$100 from Alice's account to Bob's account

Transactional file systems

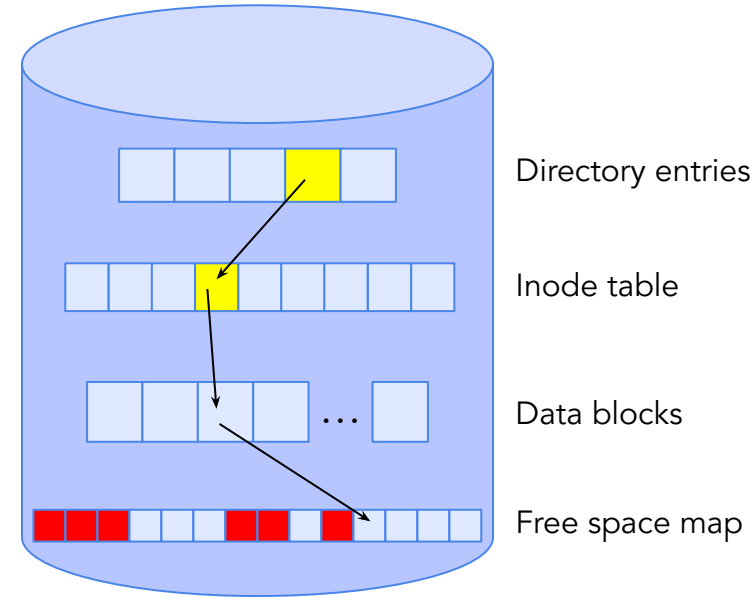
- Better reliability through use of log
 - Changes are treated as transactions
 - A transaction is committed once it is written to the log
 - Although file system may not be updated immediately, data preserved in the log
- Difference between “Log structured” and “Journaled”
 - In a Log structured filesystem, data stays in log form
 - In a Journaled filesystem, log used for recovery

Journaling file systems

- Don't modify data structures on disk directly
- Write each update as transaction recorded in a log
 - Commonly called a journal or intention list
 - Also maintained on disk (allocate blocks for it when formatting)
- Once changes are in the log, they can be safely applied to file system
 - e.g. modify inode pointers and directory mapping
- Garbage collection: once a change is applied, remove its entry from the log
- Example
 - Linux
 - Ext2: original FFS-like file system
 - Ext3: ext2 + a journal
 - Some options: whether or not to write all data to journal or just metadata
 - NTFS, Apple HFS+/apfs, Linux XFS, JFS, ext4

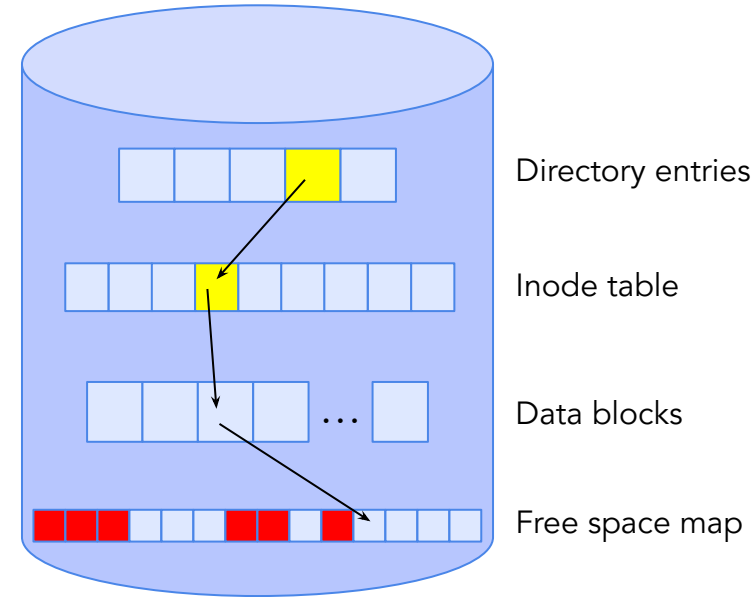
Creating a file (without journaling)

- Find free data block(s)
- Find free inode entry
- Find dirent (data structure for a directory entry)
insertion point
- -----
- Write map (i.e., mark used)
- Write inode entry to point to block(s)
- Write dirent to point to inode



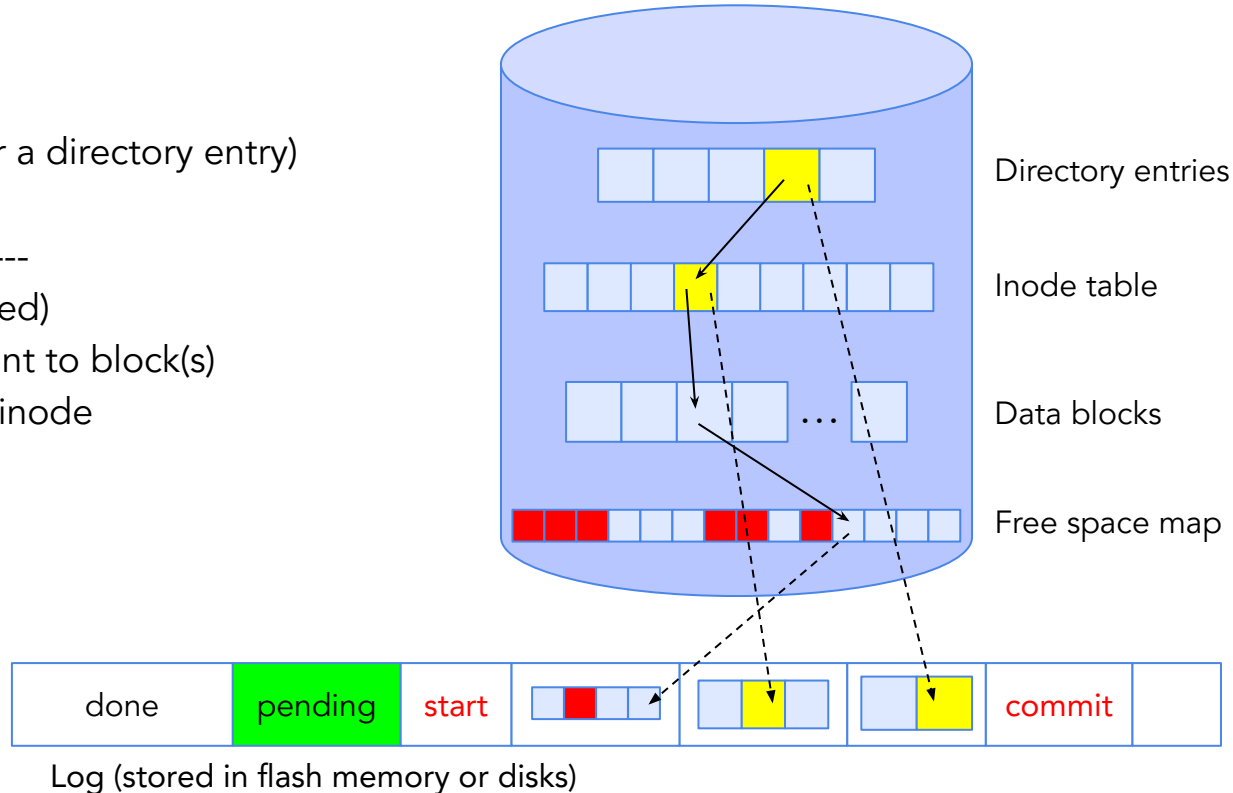
Creating a file (with journaling)

- Find free data block(s)
- Find free inode entry
- Find dirent (data structure for a directory entry) insertion point
- -----
- [log] Write map (i.e., mark used)
- [log] Write inode entry to point to block(s)
- [log] Write dirent to point to inode



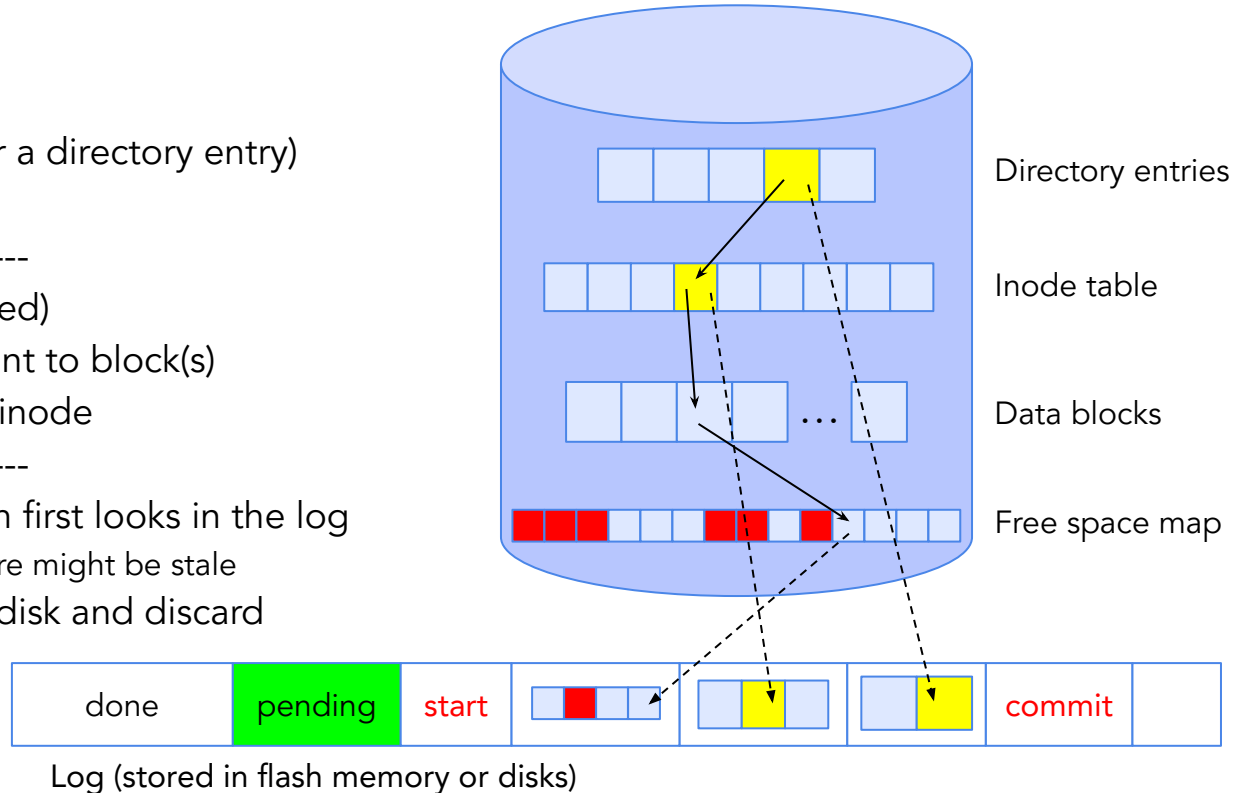
Creating a file (with journaling)

- Find free data block(s)
- Find free inode entry
- Find dirent (data structure for a directory entry)
insertion point
- -----
- [log] Write map (i.e., mark used)
- [log] Write inode entry to point to block(s)
- [log] Write dirent to point to inode



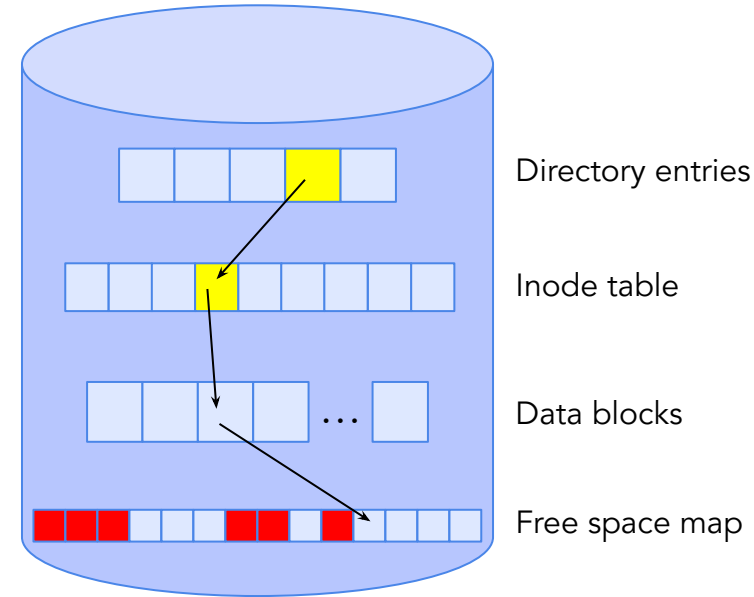
Creating a file (with journaling)

- Find free data block(s)
- Find free inode entry
- Find dirent (data structure for a directory entry) insertion point
- -----
- [log] Write map (i.e., mark used)
- [log] Write inode entry to point to block(s)
- [log] Write dirent to point to inode
- -----
- All accesses to the file system first looks in the log
 - Actual on-disk data structure might be stale
- Eventually, copy changes to disk and discard transaction from the log



Crash recovery: Discard partial transactions

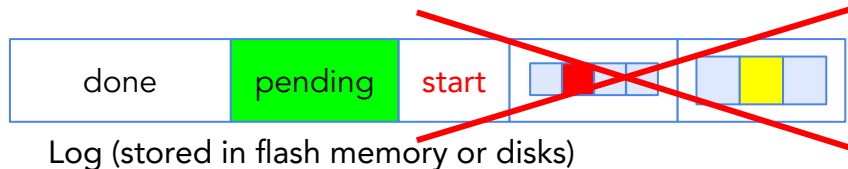
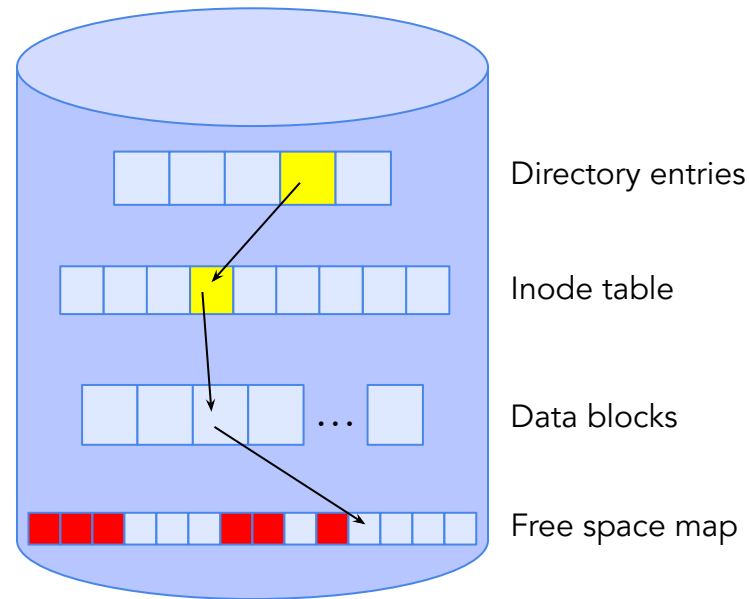
- Upon recovery
 - Scan the log
 - Detect transaction start with no commit



Log (stored in flash memory or disks)

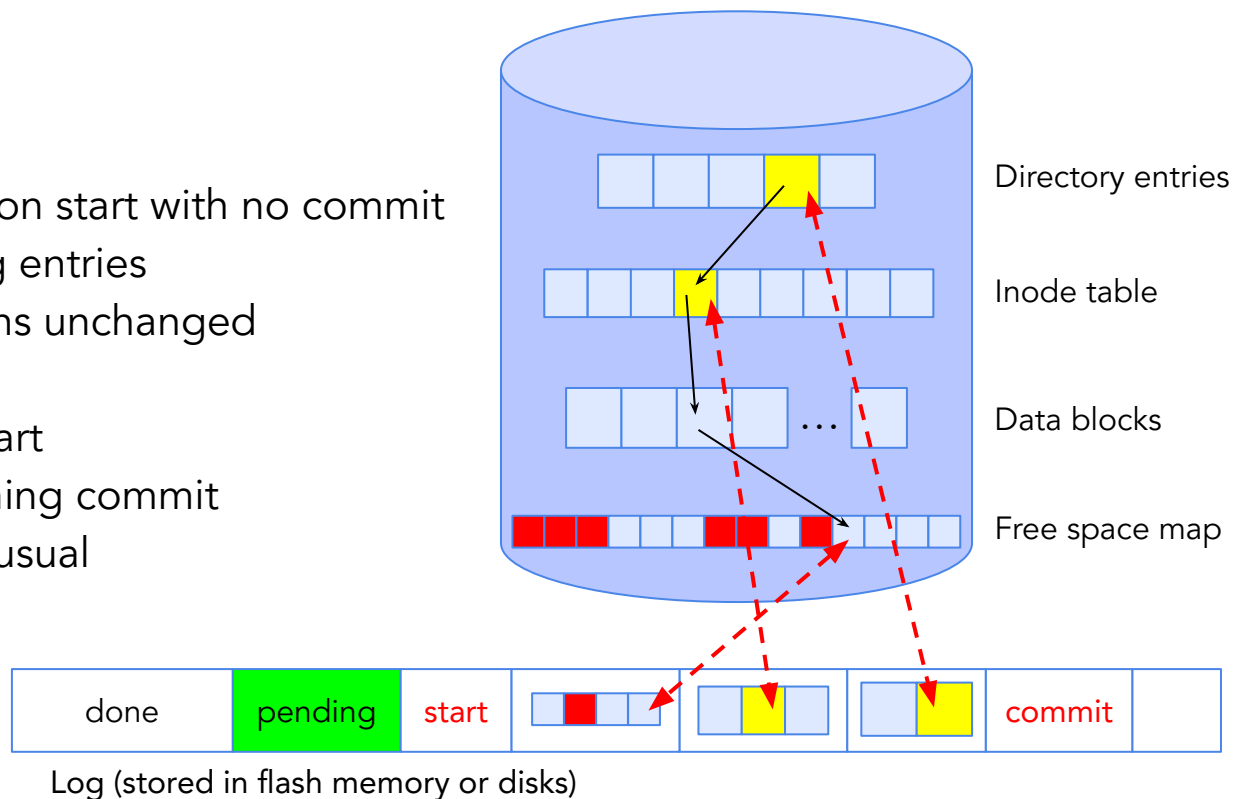
Crash recovery: Discard partial transactions

- Upon recovery
 - Scan the log
 - Detect transaction start with no commit
 - Discard log entries
 - Disk remains unchanged



Crash recovery: Keep complete transactions

- Upon recovery
 - Scan the log
 - Detect transaction start with no commit
 - Discard log entries
 - Disk remains unchanged
 - Otherwise
 - Find the start
 - Find matching commit
 - Redo it as usual



Summary

- Transaction
- Journaling
- Crash recovery